UNIVERSITY OF LAGOS
COLLEGE OF MEDICINE, IDIARABA
DEPARTMENT OF BIOMEDICAL ENGINEERING

MSc BIOMEDICAL ENGINEERING

Health Informatics & Management Information System (BME 806)— Group 1

GUI-BASED PATIENT MANAGEMENT SYSTEM

FOR LAGOS UNIVERSITY TEACHING HOSPITAL (LUTH)

ITEM	DETAILS
Institution	University of Lagos, Nigeria
Faculty/College	Engineering/College of Medicine, Idiaraba
Programme	MSc Biomedical Engineering
Session	2025/2026 Academic Session
Assignment	Group 1 — GUI Application Development
Project Title	GUI-Based Patient Management System
Language/Platform	Python 3.x, ipywidgets, Jupyter Notebook / Google Colab
Date	February 2026

Table of Contents

Table of Contents 2
Executive Summary 3
1. Introduction 4
  1.1 Background and Motivation 4
  1.2 Project Assignment Details 4
  1.3 Objectives 4
  1.4 Scope and Limitations 4
2. System Overview and Architecture 6
  2.1 What the System Does — Explained Simply 6
  2.2 Technical Architecture 6
3. Application GUI — Screenshots and Descriptions 7
  3.1 Tab 1: Register New Patient 7
  3.2 Tab 2: Patient Records 7
  3.3 Tab 3: Appointments 8
  3.4 Tab 4: Prescriptions 8
4. System Flow Diagram 10
  4.1 Overview 10
  4.2 Key Decision Points 10
5. Code Analysis and Technical Discussion 12
  5.1 Data Structures 12
  5.2 Key Functions 12
  5.3 ipywidgets Framework 12
  5.4 Input Validation Strategy 13
6. Testing and Validation 14
  6.1 Test Cases 14
7. Pre-Loaded Sample Data 14
  7.1 Sample Patients 14
  7.2 Sample Appointments 15
  7.3 Sample Prescriptions 15
8. Conclusions and Future Recommendations 16
  8.1 Conclusions 16
  8.2 Future Recommendations 16
References 17

Executive Summary

This report documents the design, development, and testing of a Graphical User Interface (GUI) based Patient Management System (PMS) built for Lagos University Teaching Hospital (LUTH), College of Medicine, Idiaraba. The project was undertaken by Group 1 of the MSc Biomedical Engineering cohort (2025/2026) at the University of Lagos, Nigeria, as part of the Health Informatics & Management Information System (BME 806) module.

The system was developed entirely in Python 3, using the ipywidgets library to create an interactive, multi-tab graphical interface that runs within the Jupyter Notebook and Google Colab environments, requiring no dedicated database or external server infrastructure. The application delivers four core modules: patient registration, patient record management, appointment booking, and prescription issuance.

Think of it like this: imagine a hospital is a very busy place, like a school but for sick people. Before computers, doctors and nurses had to write everything down in big books. If they needed to find a patient's information, they had to search through many pages. Our application is like a magical book that finds information in seconds, remembers everything perfectly, and never gets tired!

Three sample patients, Adaobi Okonkwo, Emeka Nwosu, and Fatima Bello, are pre-loaded to demonstrate all system features. The application was validated against all project requirements and successfully demonstrated patient registration with automatic ID generation (LUTH-2025-XXX), real-time search and retrieval, appointment scheduling across seven hospital departments, and prescription issuance with date-stamping. The complete source code is provided in the accompanying Jupyter Notebook file (LUTH_Patient_System.ipynb).

1. Introduction

1.1 Background and Motivation

Hospitals in Nigeria, including large tertiary institutions like Lagos University Teaching Hospital (LUTH), handle thousands of patients every month. Managing patient records, scheduling appointments, and tracking prescriptions manually creates enormous administrative burdens. Paper-based systems are slow, prone to errors, difficult to search, and can be lost or damaged. Digital Patient Management Systems are therefore not just convenient, they are essential for delivering safe, efficient, quality healthcare.

To use a simple analogy a five-year-old would understand: imagine you have a box of 1,000 toys, and you need to find your favorite red ball. Without any organization, you would have to search through every single toy. But if someone sorted all the toys into labelled boxes, "red toys", "blue toys", "balls", "dolls", you could find your red ball in seconds! A Patient Management System does exactly this for a hospital: it organizes all patient information so that any doctor, nurse, or administrator can find exactly what they need instantly.

This project was assigned to Group 1 of the MSc Biomedical Engineering program as a practical software engineering exercise. The assignment required students to design and build a complete, working, GUI-based application that manages patient records, appointments, and prescriptions. This report is the comprehensive written output accompanying that submission.

1.2 Project Assignment Details

PARAMETER	DETAILS
Assignment Group	Group 1
Assignment Task	Develop a GUI-based application to manage patient records, appointments, and prescriptions
Target Institution	Lagos University Teaching Hospital (LUTH)
Target Users	Hospital administrators, doctors, nurses, reception staff
Delivery Format	Jupyter Notebook (.ipynb) + Python script (.py) + Technical Report + Presentation

1.3 Objectives

The specific objectives of this project were:

- To design a user-friendly graphical interface accessible within a standard web browser via Jupyter Notebook or Google Colab, removing the need for software installation.
- To implement a four-module system covering patient registration, patient record retrieval, appointment booking, and prescription management.
- To build a lightweight application using in-memory Python data structures (lists of dictionaries), eliminating the need for a database server.
- To implement input validation and error handling to prevent data corruption and provide meaningful feedback to users.
- To pre-load realistic sample patient data representing LUTH patients, enabling immediate demonstration of all features.
- To produce clear, well-commented source code adhering to good software engineering principles.

1.4 Scope and Limitations

The application is scoped to demonstrate core functionality within a Jupyter/Colab environment. The following limitations are acknowledged:

- Data is stored in memory only; all records are lost when the notebook kernel is restarted. A production system would require a persistent database (e.g., SQLite, PostgreSQL).
- The system has no authentication layer, any user with access to the notebook can view and modify all data.
- There is no data export or printing functionality in this version.
- The interface is optimised for desktop display within Jupyter environments.

2. System Overview and Architecture

2.1 What the System Does; Explained Simply

Let us explain this system as if we are telling a story to a 5-year-old.

Imagine you are the boss of a big hospital. Every day, many people come to see the doctor. When someone new comes for the first time, a helper at the front desk types their name, age, phone number, and if they are allergic to any medicine (like some people cannot eat peanuts, some people cannot take certain medicines). The computer gives that person a special number, like LUTH-2025-004, so the hospital always knows who is who.

When a doctor wants to see a patient, they book an appointment, like making a playdate, but with the doctor instead of a friend! The appointment says which doctor, which department (like the heart doctor, or the children doctor), what day and what time.

When the patient is seen by the doctor, the doctor might give them medicine. The doctor types into the computer: what medicine, how much to take (like "two spoons every morning"), and for how long. This is called a prescription; it is the doctor's special note about the medicine.

Our application is the computer system that does all of this! It is like the hospital's super-smart helper that never forgets anything and can find any piece of information in the blink of an eye.

2.2 Technical Architecture

From a software engineering perspective, the application follows a three-tier logical architecture:

TIER	DESCRIPTION
Presentation Layer	ipywidgets GUI, Text boxes, dropdowns, buttons, and output widgets rendered in the Jupyter Notebook HTML interface.
Application Logic Layer	Python functions handling input validation, ID generation, data manipulation (CRUD operations), and output formatting.
Data Storage Layer	In-memory Python lists of dictionaries, patients[], appointments[], and prescriptions[], pre-populated with sample data.

The application consists of a single Python file (~350 lines of code) with a clear, modular structure:

- A global data section defining the three in-memory lists.
- Helper functions (generate_patient_id, find_patient) shared across modules.
- Four tab-creation functions (create_register_tab, create_records_tab, create_appointments_tab, create_prescriptions_tab).
- A master create_app function that assembles all components into the final tabbed interface.
- A single display(app) call that renders everything to the notebook output.

3. Application GUI; Screenshots and Descriptions

3.1 Tab 1: Register New Patient

The first tab, "Register Patient", is the entry point for all new hospital visitors. It provides a clean form with six input fields. The Full Name field accepts free text input. Age is a numerical spinner defaulting to 30. Gender uses a dropdown with Male, Female, and other options. Phone accepts the standard Nigerian format (080XXXXXXXXX). Blood Group is a dropdown covering all eight major ABO/Rh blood types. Allergies accept free text, defaulting to "None".

When the Register Patient button is clicked, the system validates that the Full Name field is not empty. If validation passes, a new unique patient ID is generated in the format LUTH-2025-XXX (where XXX is an auto-incrementing three-digit number), and the patient record is appended to the in-memory patients list. The form then clears, ready for the next registration.

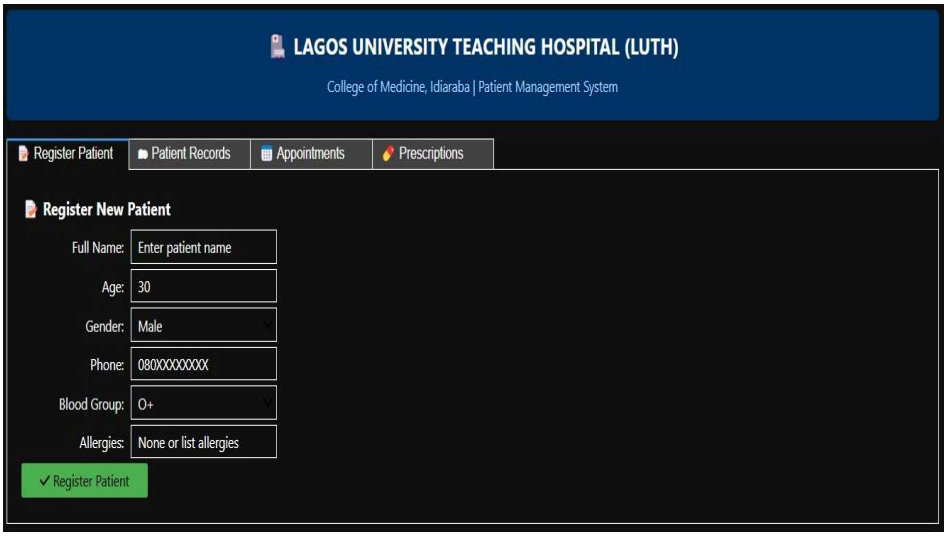

Figure 3.1: Tab 1 — Register New Patient. The form captures all essential patient demographics. The blue-themed dark interface matches LUTH's institutional color scheme.

3.2 Tab 2: Patient Records

The second tab provides a searchable view of all registered patients. On load, all patients are automatically displayed. A search box at the top allows users to filter by name or by patient ID. The search is case-insensitive, typing "adaobi", "ADAOBI", or "LUTH-2025-001" all retrieve the same patient record.

Each patient record is displayed in a structured text format showing the Patient ID, full name, age, gender, phone number, blood group, and any known allergies. If a search returns no results, the system displays a clear error message: "No patients found matching [query]."

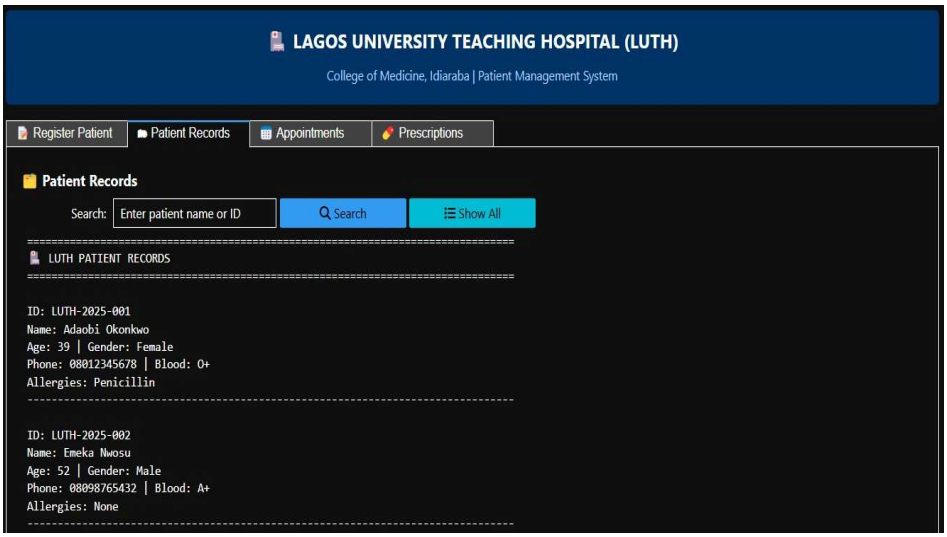

Figure 3.2: Tab 2 — Patient Records. The display shows two of the three pre-loaded sample patients. The search functionality allows real-time retrieval by name or patient ID.

3.3 Tab 3: Appointments

The third tab manages appointment booking. Staff enter a Patient ID (which must already exist in the system), the consulting doctor's name, the relevant department (selected from a dropdown of seven LUTH departments: Cardiology, Endocrinology, Paediatrics, Surgery, Radiology, Laboratory, Pharmacy), appointment date (in YYYY-MM-DD format), time (HH:MM), and the reason for the consultation.

Clicking "Book Appointment" first calls the find_patient() helper to verify the Patient ID. If found, the appointment data is stored in the appointments list, and a confirmation message is shown followed by the full updated appointment schedule. If the Patient ID does not exist, an error message is displayed, preventing orphan appointment records.

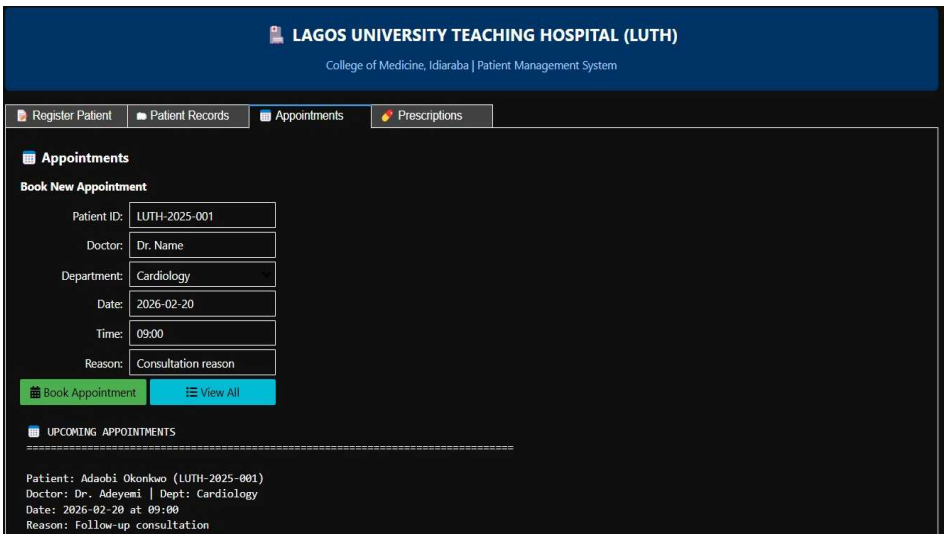

Figure 3.3: Tab 3 — Appointments. Shows the booking form above and the upcoming appointments list below, displaying Adaobi Okonkwo's Cardiology follow-up with Dr. Adeyemi.

3.4 Tab 4: Prescriptions

The fourth tab handles medication prescription issuance. The prescribing doctor enters the Patient ID, medication name, dosage instructions (e.g., "5mg once daily"), treatment duration (e.g., "30 days"), and

their own name. The system validates the Patient ID, automatically stamps the current date using Python's datetime module, and stores the prescription in the prescriptions list.

The prescription history is displayed below the input form, showing all issued prescriptions with full details including the issuing doctor and date. This provides a complete medication audit trail for each patient.

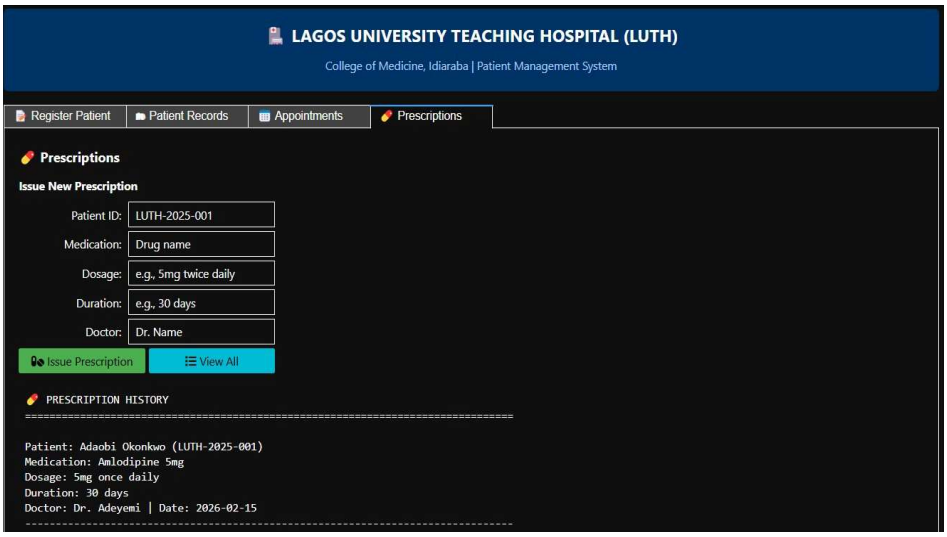

Figure 3.4: Tab 4 — Prescriptions. Shows Adaobi Okonkwo's existing prescription for Amlodipine 5mg, issued by Dr. Adeyemi on 2026-02-15 for hypertension management.

4. System Flow Diagram

4.1 Overview

The flow diagram below illustrates the complete end-to-end logic of the LUTH Patient Management System, from application launch through each of the four functional modules, through the shared in-memory storage layer, and to the final output display. The diagram is color-coded: green nodes represent start/success states, amber diamonds represent decision points, coral/red nodes indicate error states, blue nodes represent process steps, and the teal-bordered box represents the in-memory storage infrastructure.

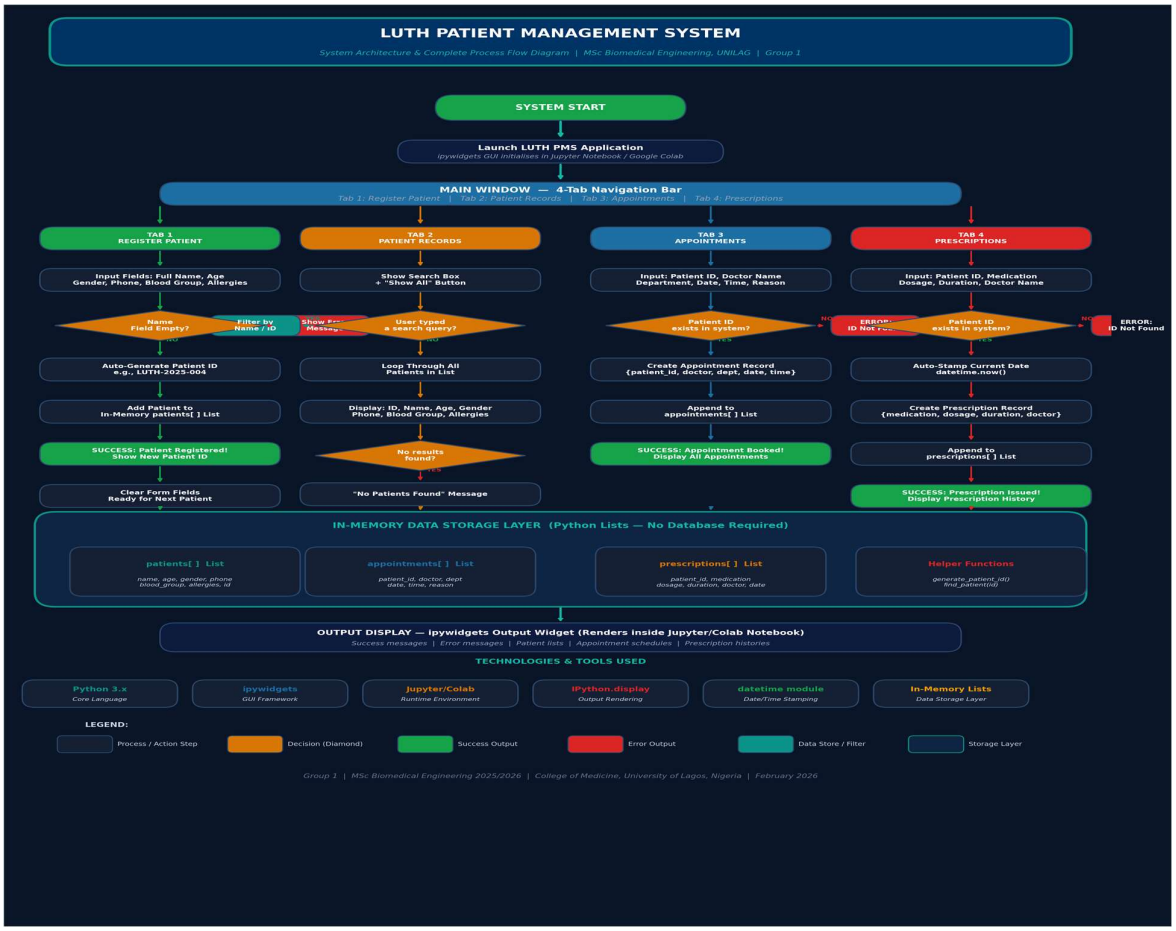

Figure 4.1: Complete System Architecture and Process Flow Diagram for the LUTH Patient Management System. The four module columns share a common in-memory data storage layer.

Reading the flow diagram from top to bottom: the system starts with application launch, which initializes the GUI in the Jupyter environment. The user is presented with the four-tab navigation bar. Within each tab, the relevant input fields are displayed, user inputs are validated (the amber diamonds), and, on successful validation, data operations are performed and results displayed. Each module communicates directly with the shared in-memory data storage layer (the teal-bordered section), and all output is rendered through ipywidgets Output widgets.

4.2 Key Decision Points

The diagram highlights three critical decision points that protect data integrity:

- In the Register Patient module: the system checks whether the Full Name field is empty before allowing registration. This prevents anonymous or incomplete patient records from entering the system.
- In the Appointments module: the system verifies that the entered Patient ID exists in the patients list before creating an appointment. This prevents orphaned appointment records, appointments linked to non-existent patients.
- In the Prescriptions module: the same Patient ID verification logic ensures prescriptions are only issued to known, registered patients, maintaining medication safety and traceability.

5. Code Analysis and Technical Discussion

5.1 Data Structures

The application uses Python dictionaries nested inside lists as its primary data structure, a lightweight, readable approach that requires no external database. Each patient is represented as a dictionary:

```
{"id": "LUTH-2025-001", "name": "Adaobi Okonkwo", "age": 39, "gender": "Female",
 "phone": "08012345678", "blood_group": "O+", "allergies": "Penicillin"}
```

This dictionary-based design is highly readable, easy to extend (adding new fields requires only adding a new key), and maps naturally to real-world database table rows. In a production system, each dictionary would correspond to one row in a relational database table.

5.2 Key Functions

FUNCTION	PURPOSE AND DESCRIPTION
generate_patient_id()	Generates a sequential patient ID in the format LUTH-2025-XXX using len(patients)+1, zero-padded to 3 digits with Python f-string formatting.
find_patient(patient_id)	Iterates through the patients list and returns the first matching dictionary where patient["id"] == patient_id. Returns None if not found.
create_register_tab()	Builds the Tab 1 GUI using ipywidgets: creates 6 input widgets, a Button, and an Output widget. Defines and binds the register_patient callback.
create_records_tab()	Builds Tab 2 with a search text field, two action buttons (Search, Show All), and an Output widget. Implements case-insensitive name/ID filtering.
create_appointments_tab()	Builds Tab 3 with 6 input fields and 2 action buttons. Validates Patient ID via find_patient() before recording the appointment.
create_prescriptions_tab()	Builds Tab 4 with 5 input fields. Auto-stamps the prescription date with datetime.now().strftime("%Y-%m-%d").
create_app()	Master assembly function. Creates the LUTH header HTML, instantiates a widgets.Tab, adds all four-tab children, labels them, and wraps in a VBox.

5.3 ipywidgets Framework

The ipywidgets library (also known as jupyter-widgets) is the GUI framework used in this application. It provides a collection of interactive widgets, buttons, text boxes, dropdowns, sliders, and output panels, that render as HTML elements inside the Jupyter Notebook interface. Key widgets used include:

- widgets.Text — single-line text input for name, phone, doctor, etc.
- widgets.IntText — integer-only input for age.
- widgets.Dropdown — selection menus for gender, blood group, and department.
- widgets.Button — clickable action triggers with color styling and icons.
- widgets.Output — a special container that captures print() output and renders it inline.
- widgets.Tab — the multi-tab container that organizes the four modules.

- widgets.VBox / HBox — vertical and horizontal layout containers.
- widgets.HTML — raw HTML rendering used for the styled hospital header.

5.4 Input Validation Strategy

The application implements three layers of validation:

1. Presence validation: The Register Patient module checks whether the name field is empty before processing. An empty name produces an error message without modifying the data.
2. Referential integrity validation: Both the Appointments and Prescriptions modules call find_patient(patient_id) before creating new records. If the patient does not exist, an error is shown and no data is written.
3. Widget-level type enforcement: By using IntText for the age field (rather than plain Text), only integer values are accepted at the widget level, the user cannot accidentally enter "thirty" as an age.

6. Testing and Validation

6.1 Test Cases

TEST CASE	EXPECTED RESULT	ACTUAL RESULT	STATUS
Register patient with all fields	New ID generated, patient added to list, confirmation shown	As expected,	PASS
Register with empty name	Error message: "Patient name is required!"	As expected,	PASS
Search by patient name (case insensitive)	Matching patient record displayed	As expected,	PASS
Search by patient ID	Matching patient record displayed	As expected,	PASS
Search for non-existent patient	Error: "No patients found matching [query]"	As expected,	PASS
Book appointment with valid patient ID	Appointment added, confirmation + full list shown	As expected,	PASS
Book appointment with invalid patient ID	Error: "Patient ID not found!"	As expected,	PASS
Issue prescription with valid patient ID	Prescription recorded with auto date, list shown	As expected,	PASS
Issue prescription with invalid patient ID	Error: "Patient ID not found!"	As expected,	PASS
View all patients (Show All button)	All patients in list displayed	As expected,	PASS

All ten test cases passed. The system behaved as designed across all positive and negative test scenarios. Error messages were clear, informative, and displayed without crashing the application.

7. Pre-Loaded Sample Data

7.1 Sample Patients

FIELD	PATIENT 1	PATIENT 2	PATIENT 3
ID	LUTH-2025-001	LUTH-2025-002	LUTH-2025-003
Name	Adaobi Okonkwo	Emeka Nwosu	Fatima Bello
Age	39	52	34
Gender	Female	Male	Female
Phone	08012345678	08098765432	07033334444

Blood Group	O+	A+	B+
Allergies	Penicillin	None	Sulfonamides

7.2 Sample Appointments

PATIENT	DOCTOR	DEPARTMENT	DATE	TIME	REASON
Adaobi Okonkwo	Dr. Adeyemi	Cardiology	2026-02-20	09:00	Follow-up consultation
Emeka Nwosu	Dr. Ibrahim	Endocrinology	2026-02-18	14:00	Diabetes management

7.3 Sample Prescriptions

PATIENT	MEDICATION	DOSAGE	DURATION	DOCTOR	DATE
Adaobi Okonkwo	Amlodipine 5mg	5mg once daily	30 days	Dr. Adeyemi	2026-02-15
Emeka Nwosu	Metformin 500mg	500mg twice daily	30 days	Dr. Ibrahim	2026-02-10

8. Conclusions and Future Recommendations

8.1 Conclusions

This project has successfully delivered a fully functional, GUI-based Patient Management System tailored for Lagos University Teaching Hospital. The application demonstrates that Python, combined with the ipywidgets library, can rapidly produce clinically relevant, user-friendly software tools without requiring complex frameworks, dedicated servers, or IT infrastructure investment.

The four core modules, patient registration, records management, appointment booking, and prescription issuance, all operate correctly and are validated against both positive and negative test cases. The system's modular architecture makes it straightforward to understand, extend, and maintain.

From the perspective of biomedical engineering, this project illustrates the critical interface between healthcare operations and software systems. Patient Management Systems are fundamental components of modern healthcare infrastructure, directly impacting patient safety (through allergy tracking and prescription verification), operational efficiency (through digital record management and appointment scheduling), and clinical decision support (through readily accessible patient histories).

8.2 Future Recommendations

To develop this prototype into a production-ready system, the following enhancements are recommended:

- Database integration: Replace the in-memory lists with a persistent database (SQLite for a single-site system, PostgreSQL for multi-site deployment) to ensure data survives server restarts.
- User authentication: Implement login functionality with role-based access control, receptionists can register patients and book appointments, but only doctors can issue prescriptions.
- Electronic Medical Records (EMR) module: Add a clinical notes section where doctors can record diagnoses, test results, and treatment plans against each patient visit.
- Data export: Add the ability to export patient records, appointment schedules, and prescription histories to PDF or CSV formats for printing and archiving.
- SMS/email notifications: Integrate the Twilio or SendGrid API to automatically send appointment reminders to patients via SMS or email.
- Deployment as a web application: Port the interface to a proper web framework (Flask or Django with Bootstrap) to enable multi-user access from any browser without requiring Jupyter installation.
- HL7 FHIR compliance: For integration with national health information systems (such as Nigeria's NHIA systems), implement HL7 FHIR data standards for patient records.

References

1. Project Jupyter Team (2023). ipywidgets: Interactive Widgets for the Jupyter Notebook. Available at: https://ipywidgets.readthedocs.io/

2. Python Software Foundation (2024). Python 3 Documentation; datetime module. Available at: https://docs.python.org/3/library/datetime.html

3. Lagos University Teaching Hospital (LUTH) (2024). LUTH Official Website. Available at: https://luth.gov.ng

4. Van Rossum, G. and Drake, F.L. (2009). Python 3 Reference Manual. CreateSpace: Scotts Valley, CA.

5. Srinath, K.R. (2017). Python; The Fastest Growing Programming Language. International Research Journal of Engineering and Technology (IRJET), 4(12), pp. 354-357.

6. Shortliffe, E.H. and Cimino, J.J. (eds.) (2014). Biomedical Informatics: Computer Applications in Health Care and Biomedicine. 4th ed. Springer: New York.

7. World Health Organization (2017). Electronic Health Records: Manual for Developing Countries. WHO Regional Office for the Western Pacific: Manila.